High Contrast Imaging for Python (HCIPy): an open-source adaptive optics and coronagraph simulator

Emiel H. Por[a], Sebastiaan Y. Haffert[a], Vikram M. Radhakrishnan[a], David S. Doelman[a], Maaike van Kooten[a], and Steven P. Bos[a]

[a]Leiden Observatory, Leiden University, P.O. Box 9513, 2300 RA Leiden, The Netherlands

ABSTRACT

HCIPy is a package written in Python for simulating the interplay between wavefront control and coronagraphic systems. By defining an element which merges values/coefficients with its sampling grid/modal basis into a single object called Field, this minimizes errors in writing the code and makes it clearer to read. HCIPy provides a monochromatic Wavefront and defines a Propagator that acts as the transformation between two wavefronts. In this way a Propagator acts as any physical part of the optical system, be it a piece of free space, a thin complex apodizer or a microlens array.

HCIPy contains Fraunhofer and Fresnel propagators through free space. It includes an implementation of a thin complex apodizer, which can modify the phase and/or amplitude of a wavefront, and forms the basis for more complicated optical elements. Included in HCIPy are wavefront errors (modal, power spectra), complex apertures (VLT, Keck or Subaru pupil), coronagraphs (Lyot, vortex or apodizing phase plate coronagraph), deformable mirrors, wavefront sensors (Shack-Hartmann, Pyramid, Zernike or phase-diversity wavefront sensor) and multi-layer atmospheric models including scintillation).

HCIPy aims to provide an easy-to-use, modular framework for wavefront control and coronagraphy on current and future telescopes, enabling rapid prototyping of the full high-contrast imaging system. Adaptive optics and coronagraphic systems can be easily extended to include more realistic physics. The package includes a complete documentation of all classes and functions, and is available as open-source software.

Keywords: HCIPy, Python, Simulations, High Contrast Imaging, Adaptive Optics, Coronagraphy, Open source

1. INTRODUCTION

During the design process of an astronomical instrument, numerical simulations of the optical system play an integral role. All components are tested separately for their functionality, and then often integrated into a complete end-to-end simulation to verify that all subsystems are able to operate seamlessly together. The rapid prototyping provided by computer simulations has sped up the development process of high contrast imaging instruments tremendously. Some examples of simulation packages for adaptive optics (AO) are: YAO,[1] OCTOPUS,[2] DASP,[3] COMPASS[4] and most recently SOAPY.[5] Some examples of packages that simulate coronagraphs are PROPER[6] and POPPY.[7] In each of these packages, either the adaptive optics system or the coronagraph is simulated and and subsystems can only be optimized one at a time.

More recently, people have started looking into optimizing the full system, rather than each element separately. For example, one can optimize the coronagraph to suppress the aberrations that are common for a specific AO system. In this method there is little interaction between coronagraph and the AO system: the AO system can be simulated first, and the resulting data can be used for optimizing the coronagraph. In other cases the interaction can be stronger. For instance, in the case of optimizing the AO system to keep a specific region of interest in the focal plane dark, rather than flattening the wavefront.[8] Another example in this category is end-to-end simulations of post-coronagraphic focal-plane wavefront sensing methods including feedback to the AO system. For examples, see this excellent review[9] and references therein. These use cases require both accurate and simultaneous simulation of both the AO system and the downstream coronagraph.

Correspondence: por@strw.leidenuniv.nl

Adaptive Optics Systems VI, edited by Laird M. Close, Laura Schreiber, Dirk Schmidt, Proc. of SPIE Vol. 10703, 1070342 · © 2018 SPIE
CCC code: 0277-786X/18/$18 · doi: 10.1117/12.2314407

These aforementioned type of simulations place a particular set of requirements on the simulation software. The software needs to give baseline performance, to allow the user to simulate part of the system that he/she is not interested in with minimal effort, while still being modular to allow for inclusion of completely new components. The following guiding principles were used during the design process of HCIPy:

- *Modularity.* Components should be written to work independently of each other. For example, in this way, wavefront sensors can be exchanged, coronagraphs replaced and optical elements be made more realistic, without having any influence on the operation of the other systems. This allows for rapid prototyping of new components, such as coronagraphs or wavefront sensors, and to compare different components as equally as possible.
- *User friendliness.* Components should be written to have extensive and sensible default parameters. This provides out-of-the-box baseline performance, meaning that even inexperienced users in one of the fields can code a working system.
- *Error avoidance.* Common user errors should be hard to make. That is, HCIPy is designed to handle many mathematical details, such as sampling requirements, automatically and in the background, providing clean and readable code. This allows the user to focus on system architecture rather than the details. HCIPy allows for access to these mathematical details if necessary, but their explicit nature makes mistakes easier to catch.
- *Pythonic.* HCIPy is written in the Python language, an interpreted high-level programming language. This programming language emphasizes code readability, allows for object-oriented programming, and is in use by many astrophysical projects.[10]

In Section 2 we discuss the core functionality of HCIPy. In Sections 3, 4, 5 and 6 we explore the functionality of HCIPy further, discussing optical systems, adaptive optics, coronagraph and more. We conclude with Section 7, and look towards future work.

2. CORE FUNCTIONALITY

The main mechanism in HCIPy for following these design principles is the use of `Field`s throughout the code base. These objects behave like a sampled physical field, sampling the value of a physical quantity in space. A `Field` contains a `Grid`, which is used to define the positions of the points in space, on which a `Field` is sampled. A `Coords` object in turn defines the values of the coordinates in a `Grid` object, while not containing any information on how to interpret these coordinates.

While the concepts of `Coords`, `Grid`s and `Field`s might seem esoteric and cumbersome, in most cases the user doesn't have to interact with these classes directly. Instead, the user interacts with functions to create and modify them, and only uses the classes explicitly when he/she needs more control over the sampling for a specific part of their code. The following subsections present the implementation details of these three objects and explore the flexibility when programming with these objects.

2.1 Coords, Grids and Fields

An object of a `Coords` class can yield values for each dimension for each point in a `Grid`. Indexing is done using a single value, rather than one for each dimension. This is done to support `UnstructuredCoords`, which is a set of points with no internal structure. A `SeparatedCoords` object constrains coordinates to be a tensor product of the discretization along each axis. Some mathematical operations, such as Fourier transforms, can be performed much quicker when coordinate axes can be separated. Finally, a `RegularCoords` object constrains regularly-spaced coordinates on all axes. Fourier transforms on such a grid can use a Fast Fourier Transform (FFT) which greatly reduces computation times. Each of these `Coords` classes calculates their values on demand, rather than storing them explicitly. This greatly reduces the amount of required computer memory, especially for larger regularly-spaced grids.

A `Grid` object can be thought of as a discretization of some vector space. It contains a `Coords` class and applies physical meaning to the coordinate values by introducing a coordinate system. Additionally it can provide weights to each of the points: the interval, area, volume or hypervolume that a point covers in its vector space. These weights simplify calculation of integrals and derivatives. HCIPy supports a `CartesianGrid` for Cartesian spaces of any dimension, `PolarGrid` for polar coordinates and `SphericalGrid` for spherical coordinates. Conversion between coordinate systems can be performed by calling the `Grid.as_()` function with the required coordinate system. In addition, `Grid`s can be scaled and shifted by calling their respective functions.

A `Field` object is the discretized version of a physical field. It contains a `Grid` and an array of values. HCIPy supports scalar fields (e.g. an intensity field), vector fields (e.g. a vector electric field) and tensor fields of any dimension (e.g. a Jones matrix field, or a Mueller matrix field). Note that `Field`s in HCIPy do not have to be particle field, but simply a quantity that has a value for each point in space. For a scalar `Field` there is only a single spatial axis in the value array, independent of the dimension of the vector space, as is usually done. The dimensionality is hidden by the grid, rather than the value array. This may seem impractical and unjustified; nevertheless this is the only way to consistently handle all types of `Coords`, in particular the `UnstructuredGrid`. Moreover, this strengthens the idea that a `Field` can be viewed as a covariant vector. For a vector `Field` the value array is two dimensional, the last axis being the spatial axis, and for tensor `Field`s the value array is n-dimensional for an $(n-1)$-dimensional tensor, again the last axis being the spatial axis. Special functions exist to make handing of vector and tensor fields easier, including but not limited to a `field_dot()` function, which multiplexes a dot product over the spatial axis, a `field_inv`, which takes the inverse of a two-dimensional tensor field for each point separately, and a `field_einsum`, which can calculate all Einstein-summation-convention formulas multiplexed over the spatial axis.

2.2 Field generators and visualization

We also use the concept of "field generators", which is a function that accepts a `Grid` as its sole argument and returns a `Field` on that `Grid`. This is in places where an analytical function needs to be evaluated on a `Grid` at another position in the code. A prime example is telescope apertures, which can be evaluated on any `Grid`, and can be user defined.

To make visualization of `Field`s easier, HCIPy includes several functions mimicking, and working in conjunction with, the "Matplotlib" plotting library.[11] The most versatile function is `field_imshow`, which can draw two-dimensional scalar fields. `Field`s with `UnstructuredCoords` are automatically interpolated to be able to draw them. Complex scalar fields are drawn using a custom two-dimensional color scale, which is extremely useful to be able to see both phase and amplitude in electric field plots. Masking of unused parts of the image is supported, and useful for phase plots on a pupil. Additional functions for contour plots for `Field`s are also implemented.

An example script in how to construct and transform `Grid`s, evaluate Field generators, and display `Field`s can be found in Listing 1, along with its output in Figure 1.

2.3 Fourier transforms

Fourier transform objects act on a `Field`, and return its Fourier transformed version, on the correct `Grid` corresponding to frequency space. HCIPy implements a `NaiveFourierTransform`, which naively implements the full Fourier integral, a `FastFourierTransform`, which uses the FFT algorithm and acts on regularly-spaced grids, and a `MatrixFourierTransform`, which uses a discrete-time Fourier transform, also known as a matrix Fourier transform,[12] and acts on any separated coordinates, including regularly-spaced grids. These three classes can be used directly or HCIPy can automatically choose which type of Fourier transform is fastest on the required input and output `Grid`s, and use that one.

2.4 Mode bases

Similar to field generators, HCIPy implements a wide range of mode bases, among others the Zernike modes, Fourier modes and disk harmonics. A `ModeBasis` is a list of objects, that form a linear mode basis. The modes of a `ModeBasis` can be easily put into a matrix, providing easy access to projection and deprojection of a function onto the mode basis. A code example for constructing several mode bases can be found in Listing 2 along with its output in Figure 2.

```python
# Import packages
from hcipy import *
import matplotlib.pyplot as plt
import numpy as np

# Make a separated polar grid
r = np.logspace(-1, 1, 11)
theta = np.linspace(0, 2*np.pi, 11)
coords = SeparatedCoords((r, theta))
polar_grid = PolarGrid(coords)
cartesian_grid = polar_grid.as_('cartesian')
plt.plot(cartesian_grid.x, cartesian_grid.y, '.')
plt.show()

# Create and plot Field
pupil_grid = make_pupil_grid(1024)
aperture = make_magellan_aperture(True)
evaluated_aperture = evaluate_supersampled(aperture, pupil_grid, 8)
imshow_field(evaluated_aperture)
plt.show()
```

Listing 1: An example code showing how to create, manipulate and show fields. A more detailed guide can be found in the online documentation for HCIPy.

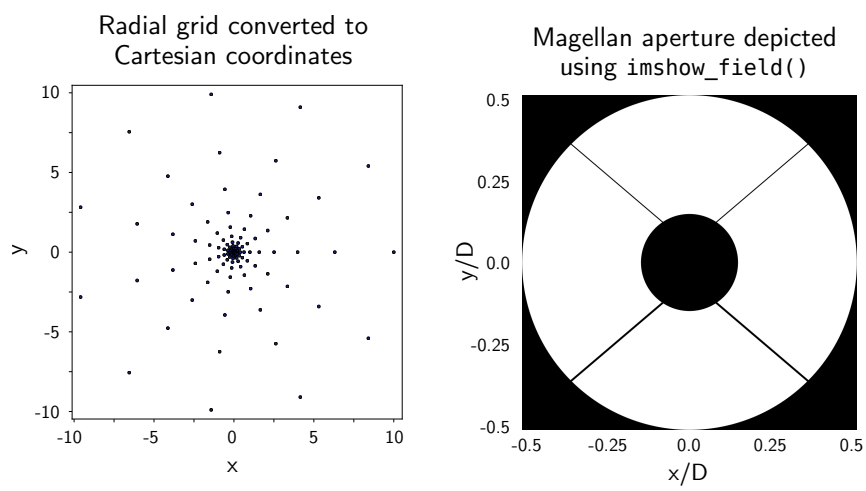

Figure 1: The resulting plots from the code in Listing 1. (*left*) The points for a separated grid using polar coordinates. (*right*) The Magellan aperture evaluated using supersampling on a pupil grid.

3. OPTICAL SYSTEMS

`Wavefronts` represent a monochromatic optical wavefront and combine an electric field with a wavelength. To support polarization, the electric field can be both scalar and vectorial, being a two-dimensional Jones vector or a full three-dimensional electric field. `Wavefronts` support many methods to make visualization easier, such as direct access to the phase, amplitude, intensity and power of the electric field.

These `Wavefronts` can be propagated through an optical system using `OpticalElements`. An `OpticalSystem` can represent any physical optical element, such as `Apodizers`, `SurfaceAberrations`, `MicrolensArrays` and even `DeformableMirrors`. All `OpticalElements` support both forward and inverse propagation, as well as calculation

```python
grid = make_pupil_grid(256)
zernike_modes = make_zernike_basis(num_modes=16, D=1, grid=grid)

for i, m in enumerate(zernike_modes):
    plt.subplot(4, 4, i + 1)
    imshow_field(m, cmap='RdBu')
plt.show()

gaussian_hermite_modes = make_gaussian_hermite_modes(grid, num_modes=16,
↪   mode_field_diameter=0.3)
gaussian_laguerre_modes = make_gaussian_laguerre_modes(grid, pmax=5, lmax=2,
↪   mode_field_diameter=0.3)
```

Listing 2: An example code demonstrate the implementation of mode bases in HCIPy. A more detailed guide can be found in the online documentation.

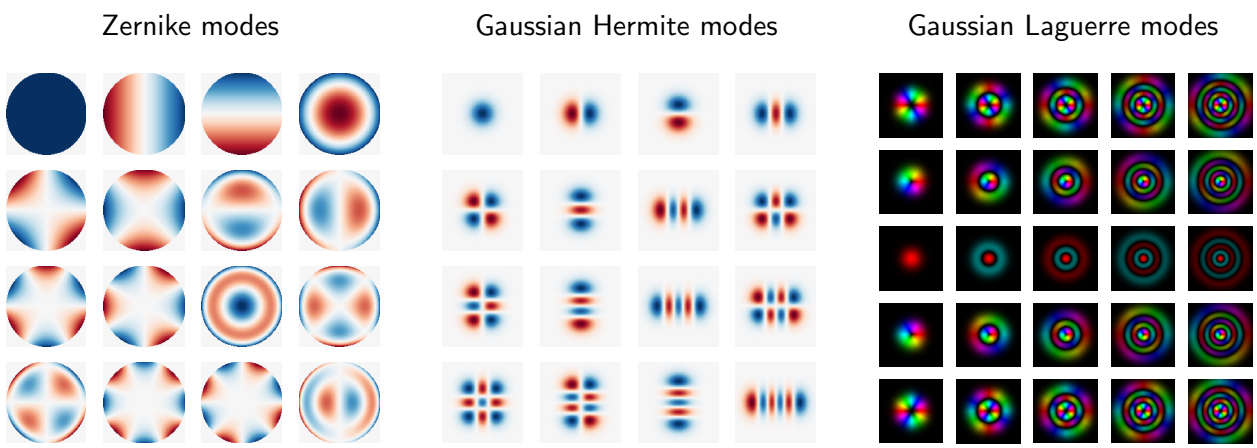

Figure 2: Some of the supported mode bases in HCIPy. This is the result of Listing 2.

of their transformation matrix, ie. the matrix of the linear transformation from input to output planes. The latter is extremely useful for post-coronagraphic wavefront control calculations and coronagraph optimization.

Propagations through space are also `OpticalElements`, specifically `Propagators`. Currently implemented are a `FresnelPropagator`, which is a near-field, paraxial propagator using an angular-spectrum method, and a `FraunhoferPropagator`, which is a far-field, paraxial propagator, which can be used to simulate lenses of specific focal lengths.

Light can be detected by `Detectors`, which can simulate camera defects and noise. A `NoisyDetector` simulates simple, empirical noise on the detector images, consisting of a flat field, dark current, photon noise and read noise. More complicated noise models are under consideration. An image from a detector can be processed using a `FrameCorrector`. This object performs dark and flat field corrections to any incoming image.

To support changing optics, a `DynamicOpticalSystem` class is available. This class allows for sub-sampled temporal integrations to allow for a fractional number of frames lag in extreme adaptive optics simulations. Updates to the optical system are scheduled and the light is internally propagated through the system. These updates are for example a changing atmosphere, a change in actuator positions for a deformable mirror, or readout of a detector. This class forms the basis for an AO system class.

4. ADAPTIVE OPTICS

4.1 Atmospheric modeling

To simulate the atmosphere above the telescope, HCIPy uses infinitely-thin phase screens, implemented as `AtmosphericLayer` objects. These phase screens can move over the telescope aperture, according to the "Frozen Flow" approximation.[13] The phase pattern can be generated in two different ways, extendable by the user. Subharmonics[14] can be added by using a multi-scale Fourier transform method by `FiniteAtmosphericLayer`. This yields excellent results for short-time simulations, where the phase screen doesn't move that much. For longer simulations, a `InfiniteAtmosphericLayer` is used, which extrudes a long ribbon of phase screen by adding rows sequentially, based on previous samples.[15] This method can also simulate non-stationary turbulence.

These single layers can be combined into a `MultiLayerAtmosphere` object. This object simulated the propagation between individual layers using Fresnel propagation, which can simulate scintillation. An example script introducing how to construct and use atmospheric layers and models can be found in Listing 3, along with an example output in Figure 3.

```
# Create an atmospheric layer
layer = InfiniteAtmosphericLayer(pupil_grid, Cn_squared, L0, velocity, height)

# Propagate a wavefront through the layer
wf = layer(wf)

imshow_field(wf.phase, cmap='RdBu')
plt.show()

# Make a multi-layer atmosphere
layer = make_standard_atmospheric_layers(pupil_grid, L0)
atmos = MultiLayerAtmosphere(layers, scintillation=True)

wf = atmos(wf)
imshow_field(wf.intensity)
plt.show()
```

Listing 3: An example code showing the creation of an atmospheric model including scintillation. A more detailed guide can be found in the online documentation for HCIPy.

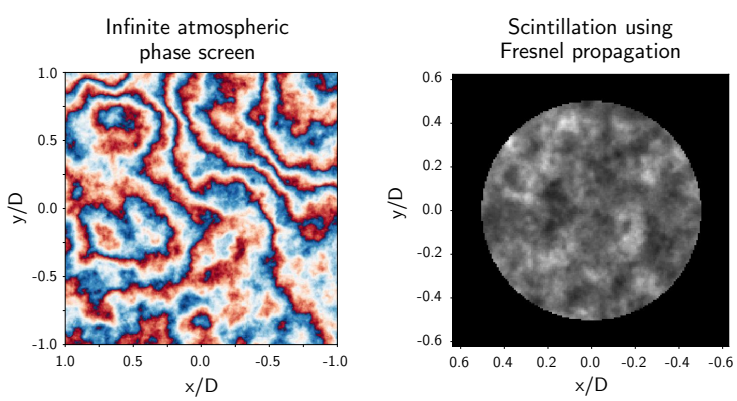

Figure 3: The simulated images from Listing 3. These images show a simulated phase screen that can be extruded infinitely in any direction (*left*), and the scintillation from a series of phase screens with Fresnel propagation between layers (*right*).

4.2 Wavefront sensing

Wavefront sensing is done using four operations from different classes. A `WavefrontSensorOptics` class simulates the optics in front of the detector for a certain wavefront sensor. Afterwards, a `Detector` class is used to get the wavefront sensor image. This image can be corrected for flat-field and dark effects using a `FrameCorrector` object. Finally, a `WavefrontSensorEstimator` is used to reduce the corrected detector image into an estimate of the wavefront. This can be either slopes, or actual wavefront, or anything that varies with wavefront. A `WavefrontSensor` object is available for folding all the different steps into a single object. This separation into optics and estimator makes it possible to play back and estimate wavefronts from real wavefront sensor images. An example script on how to construct and use a Pyramid wavefront sensor is presented in Listing 4.

HCIpy supports several wavefront sensors: among others, a Shack-Hartmann, Pyramid[16,17] (only unmodulated), Zernike[18,19] and (generalized) optical-differentiation wavefront sensor.[20,21] Example detector images are shown for each of these wavefront sensors in Figure 4.

4.3 Wavefront control

All wavefront controllers are derived from a base `Controller` class. This class can transform measured wavefront sensor estimates from one or more `WavefrontSensorEstimator`s into actuator voltages for one or more `DeformableMirror`s. We will implement an integral controller, which uses a (leaky) integrator, and a full PID controller.

The wavefront sensor estimates can be optionally filtered by an `Observer`, which allows for separation of estimation and control. We will implement a modal reconstructor, which reconstructs modes using a linear transformation matrix, a Kalman filter and a linear minimum mean squared error (LMMSE) estimator. All observers are designed to be used outside of wavefront control as well.

5. CORONAGRAPHY

Coronagraphs are implemented as optical elements, most of them taking a pupil-plane `Wavefront` as input and outputting a post-coronagraphic pupil-plane `Wavefront`. A `LyotCoronagraph` and `OccultedLyotCoronagraph` implement both cases of Lyot coronagraphs. The first implements a Lyot coronagraph with a small occulting mask size, and assumes that the part outside of the focal-plane mask is transmitted. The second case assumes that the part outside of the focal-plane mask is occulted. Both of these are in use by, for example, the shaped-pupil Lyot coronagraph. The focal-plane and Lyot-stop masks can be replaced by realistic `OpticalElements` to more accurately describe a specific implementation of a Lyot-style coronagraph.

The `VortexCoronagraph` class implements a propagation through an optical vortex.[22,23] It uses a multi-scale optical propagation scheme to resolve the vortex singularity. Functions for generating pupil and Lyot-stop masks for the ring-apodized vortex coronagraph[24] are implemented.

The `ApodizingPhasePlateCoronagraph` implements an APP coronagraph.[25–27] Several functions for optimizing this coronagraph, ie. calculating a phase pattern that creates a dark zone in a region of interest in the focal plane, are implemented. These methods include a quick-and-versatile method based on a modified Gerchberg-Saxton algorithm by Christoph Keller (in prep.), and a globally optimal linear optimization method.[28] The latter of these requires the installation of Gurobi[29] including its Python interface.

A `ShapedPupilCoronagraph` is also available, including methods for optimizing those, based on global linear optimization.[30] The same function can optimize shaped-pupil Lyot coronagraphs[31] as well, for fixed focal-plane and Lyot-stop masks.

A `PerfectCoronagraph` is also implemented. The implementation is based on fitting and subsequent subtraction of the electric field of an unaberrated PSF.[32] Higher-order perfect coronagraphs are implemented as well using a similar method.[33] These coronagraphs are highly computationally efficient and can be used for quick comparisons with other coronagraphs, or if a specific coronagraph model is not needed.

```python
# Create the optics for the WFS
wfso = PyramidWavefrontSensorOptics(pupil_grid, pupil_separation=1.5, num_pupil_pixels=64, refractive_index=1.5)

# Create detector model
detector = NoisyDetector(input_grid=wfso.output_grid, flat_field=0.01)

# Create the frame corrector
frame_corrector = BasicFrameCorrector(flat_field=detector.flat_field)

# Create the WFS estimator
wfse = PyramidWavefrontSensorEstimator(aperture=circular_aperture(1), output_grid=wfso.output_grid)

# Combine all into a single object
wfs = WavefrontSensor(wfso, detector, frame_corrector, wfse)

# Measure an incoming wavefront using a 1sec exposure
wfs.integrate(incoming_wavefront, 1)
slopes = wfs.read_out()
```

Listing 4: An example code showing the setup and reading out of an unmodulated Pyramid wavefront sensor. A more detailed guide can be found in the online documentation for HCIPy.

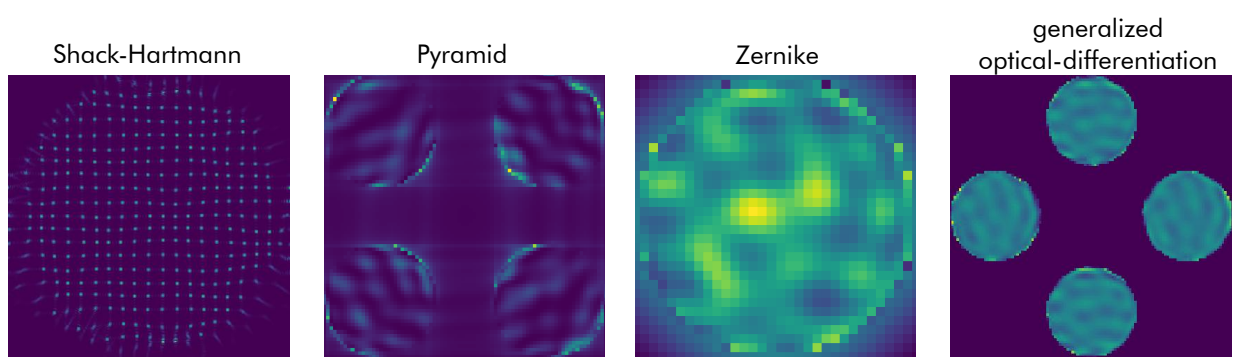

Figure 4: Simulated images for different wavefront sensors using HCIPy. All wavefront sensors see the same phase aberration. The amplitude of the aberration was scaled to approximately the linear range of the wavefront sensor, for visualization purposes.

```python
# Create the pupil
pupil_grid = make_pupil_grid(2048, 4)
pup = circular_aperture(1)(pupil_grid)

# Create a vortex coronagraph
coro = VortexCoronagraph(pupil_grid, charge=4)

# Create some aberrations
aber = SurfaceAberration(pupil_grid, ptp=1/8, diameter=1)

# Create a Lyot stop mask
lyot = lambda grid: circular_aperture(0.95)(grid) + circular_aperture(0.05, [1.8, 0])(grid)
lyot = Apodizer(lyot(pupil_grid))

# Create post-coronagraphic image
img = prop(lyot(coro(aber(Wavefront(pup)))))
```

Listing 5: An example code showing the simulation of an image for a self-coherent camera[34] (SCC) behind a charge 4 vortex coronagraph. A more detailed guide can be found in the online documentation.

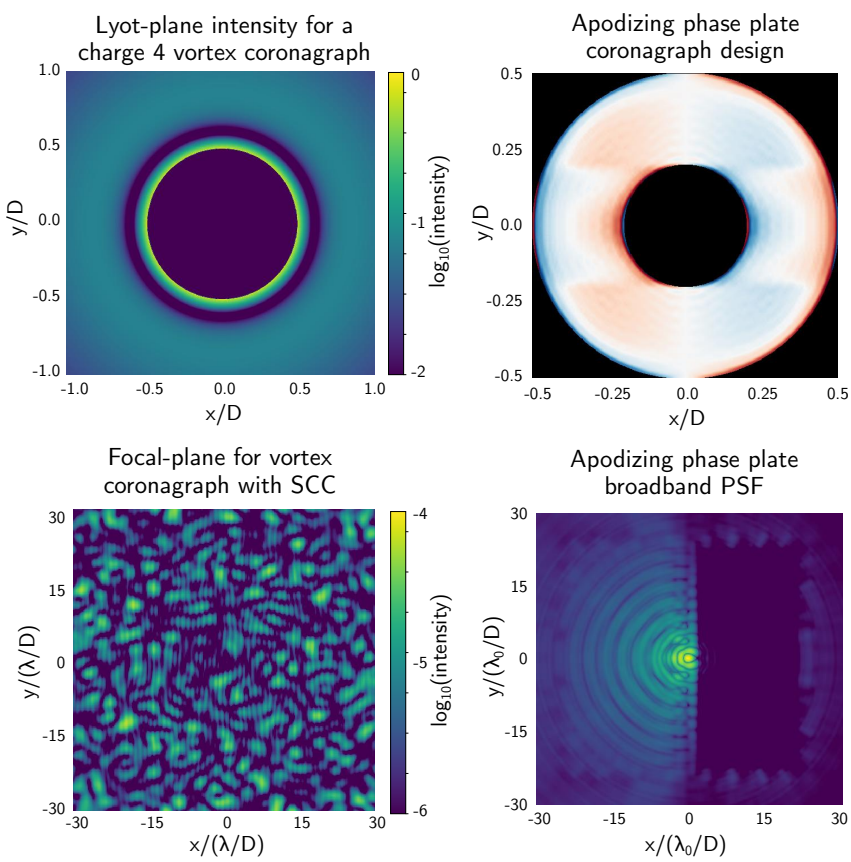

Figure 5: A series of simulated coronagraphic images with HCIPy. (*top left*) The Lyot-plane field for a charge 4 vortex coronagraph. (*bottom left*) The focal-plane image of the vortex coronagraph, with a self-coherent camera[34] Lyot-stop mask. The code for generating this image can be found in Listing 5. (*top right*) An example apodizing phase plate design optimized using HCIPy. (*bottom right*) The broadband point spread function for this APP design.

6. MISCELLANEOUS

6.1 Polarization

Polarization is supported in HCIPy using Jones calculus. This includes the implementation of polarizers and waveplates with (spatially) varying fast-axis orientation, retardance and circularity. This has allowed for research into broadband vector Apodizing Phase Plates for polarimetry and polarization aberrations, see Figure 6.

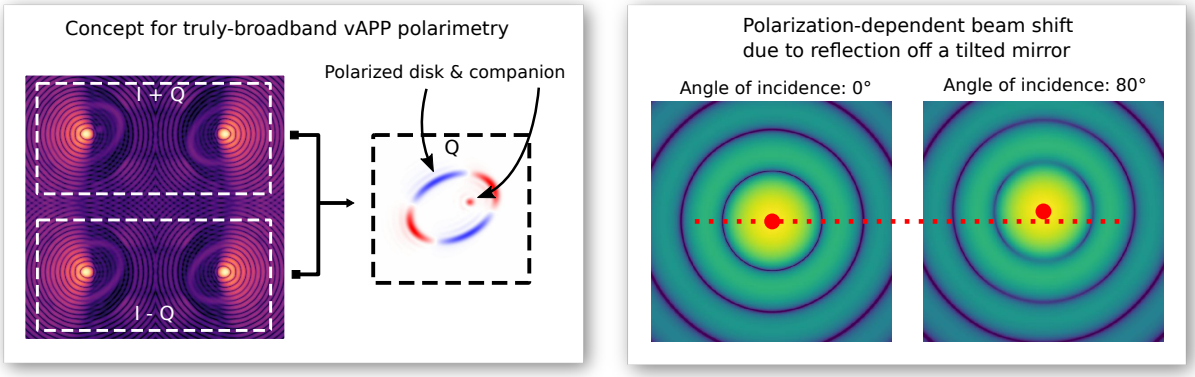

Figure 6: Some examples of polarization effects that can be simulated. (*left*) A truly-broadband vAPP implementation for polarimetry. A double difference detection scheme is simulated. This figure is adapted from Bos et al.[35] (*right*) The effect of polarization due to reflection off a tilted aluminum-coated mirror. This figure is courtesy of Rob van Holstein, work to appear in Van Holstein et al. in prep..

6.2 Performance

While computational performance is not the main goal of HCIPy, care has been taken to retain as much performance as possible. Algorithmic improvements are always preferred in cases where they don't compromise on readability and/or generality. This can be seen in the judicious usage of the Matrix Fourier transform, wherever possible. When performing a Fourier transform, HCIPy automatically determines whether a Matrix Fourier transform or a zero-padded Fast Fourier transform would yield the shortest computation time.

Care has been taken with the numerical efficiency as well. The library "NumPy" is used for arrays and simple linear algebra operations.[36] The library "SciPy" is used for more intricate linear algebra.[37] The library "PyFFTW" is used to calculate FFTs using the well-known FFTW library.[38] HCIPy does not internally use multiprocessing or multithreading yet, apart from the parallelization by linear algebra packages and FFTW. This seemingly strange design decision has its basis in the level at which to parallelize. It is much easier and computationally efficient to parallelize at the highest level possible, as in most cases the workload is embarrassingly parallel. Think for instance at multiplexing different wavelengths for a broadband simulation, or the different parameter sets in a parameter study. It is much harder to efficiently parallelize the Fast Fourier transform. The task of parallelizing is therefore left to the user: he/she now has to write the code for parallelization. GPU support is under development.

7. CONCLUSIONS

7.1 Overview

HCIPy is an object-oriented framework written in Python for performing end-to-end simulations of high-contrast imaging instruments. It is built around the concept of Fields, a sampled version of physical fields. This union allows the user to focus on the high-level structure of their own code, rather than worry about sampling details.

HCIPy defines wavefronts, optical elements and optical systems. Propagators are optical elements that are used for propagation through free space; HCIPy implements both Fraunhofer and Fresnel diffraction propagators. Jones calculus is used for polarization calculations, with polarizers and waveplates supported out of the box.

Atmospheric turbulence is simulated using thin infinitely-long phase screens. Scintillation is modeled using Fresnel propagation between individual layers. Implemented wavefront sensors include the Shack-Hartmann and Pyramid wavefront sensors. For coronagraphy the vortex, Lyot and APP coronagraphs are implemented, with methods for globally optimizing pupil-planes, both in phase and amplitude, based on linear optimization.

As the AO and coronagraphy can be used in series in the same software package, HCIPy allows for end-to-end simulations of post-coronagraphic focal-plane wavefront sensing including feedback and implementation of AO control algorithms specifically designed for high-contrast imaging applications. HCIPy is available as open-source software on GitHub at https://github.com/ehpor/hcipy, and is licensed under the MIT license.

7.2 Future plans

The following subjects are actively considered or in development:

- *GPGPU support.* With the advances in the processing power contained in Graphics Processing Units (GPUs) and the almost universal support for General Purpose computations on GPUs,[39,40] support for GPGPU cannot be neglected in any major simulation package. Especially the GPUs affinity for linear algebra operations cannot be ignored. GPGPU support is currently under development for HCIPy. As most heavy numerical calculations are performed on Fields, it makes sense to seamlessly do calculations on Fields in the background on the GPU.

- *Data reduction.* With ground-based observations post-processing techniques play an important role. This cannot be neglected in simulations: we would want to optimize our AO system and coronagraph for post-processed contrast rather than pre-processed contrast. Data-reduction techniques will preferably be included in HCIPy with wrappers to more advanced and mature data-reduction packages such as pyKLIP[41] and VIP.[42]

- *Improved detector modeling.* Noise behavior of detectors have a significant influence on the AO system performance, especially at low flux levels. Improved detector models, such as an (empirical) CCD and CMOS,[43] or EMCCD[44] noise model would be helpful in characterizing these effects.

- *Non-paraxial vector diffraction.* Taking into account the vector nature of light goes further than just supporting polarization-sensitive optical elements: the propagation of light needs to be changed as well, especially in low F-number beams where non-paraxial effects play a much larger role.[45]

- *More wavefront sensors.* Support for more wavefront sensors is always valued. Currently the coronagraphic Modal Wavefront Sensor[46] and phase diversity[47] are planned.

Community input and extensions are warmly appreciated.

ACKNOWLEDGMENTS

EP was responsible for the inception of the project, full design and the majority of the development effort. SH provided the Pyramid, Optical-Differentiation and Zernike wavefront sensors. VR contributed to the Shack-Hartmann wavefront sensor implementation. DD provided field generators for realistic telescope apertures. MK contributed to the adaptive optics design and implemented the adaptive optics control code. SB provided the noisy detector model.

EP acknowledges funding by The Netherlands Organisation for Scientific Research (NWO) and the São Paulo Research Foundation (FAPESP).

REFERENCES

[1] Rigaut, F., "Yao adaptive optics simulation package," (2002).

[2] Le Louarn, M., Vérinaud, C., Korkiakoski, V., Hubin, N., and Marchetti, E., "Adaptive optics simulations for the european extremely large telescope," in [*Advances in Adaptive Optics II*], **6272**, 627234, International Society for Optics and Photonics (2006).

[3] Basden, A., Geng, D., Myers, R., and Younger, E., "Durham adaptive optics real-time controller," *Applied optics* **49**(32), 6354–6363 (2010).

[4] Gratadour, D., Puech, M., Vérinaud, C., Kestener, P., Gray, M., Petit, C., Brulé, J., Clénet, Y., Ferreira, F., Gendron, E., et al., "Compass: an efficient, scalable and versatile numerical platform for the development of elt ao systems," in [*Adaptive Optics Systems IV*], **9148**, 91486O, International Society for Optics and Photonics (2014).

[5] Reeves, A., "Soapy: an adaptive optics simulation written purely in python for rapid concept development," in [*Adaptive Optics Systems V*], **9909**, 99097F, International Society for Optics and Photonics (2016).

[6] Krist, J. E., "Proper: an optical propagation library for idl," in [*Optical Modeling and Performance Predictions III*], **6675**, 66750P, International Society for Optics and Photonics (2007).

[7] Perrin, M. D., Soummer, R., Elliott, E. M., Lallo, M. D., and Sivaramakrishnan, A., "Simulating point spread functions for the James Webb Space Telescope with WebbPSF," in [*Space Telescopes and Instrumentation 2012: Optical, Infrared, and Millimeter Wave*], *Proc. SPIE* **8442**, 84423D (Sept. 2012).

[8] Radhakrishnan, V. M., Doelman, N., and Keller, C. U., "Optimization of contrast in adaptive optics for exoplanet imaging," in [*Adaptive Optics Systems VI*], *Proc. SPIE* **10703** (2018).

[9] Jovanovic, N., Absil, O., Baudoz, P., Beaulieu, M., Bottom, M., Cady, E., Carlomagno, B., Carlotti, A., Doelman, D., Fogarty, K., Galicher, R., Guyon, O., Haffert, S., Huby, E., Jewell, J., Keller, C., Kenworthy, M. A., Knight, J., Kühn, J., Miller, K., Mazoyer, J., N'Diaye, M., Por, E., Pueyo, L., Riggs, A. J. E., Ruane, G., Sirbu, D., Snik, F., Wallace, J. K., Wilby, M., and Ygouf, M., "Review of high-contrast imaging systems for current and future ground-based and space-based telescopes II. Common path wavefront sensing/control and Coherent Differential Imaging," in [*Adaptive Optics Systems VI*], *Proc. SPIE* **10703** (2018).

[10] The Astropy Collaboration et al., "The Astropy Project: Building an inclusive, open-science project and status of the v2.0 core package," *ArXiv e-prints* (Jan. 2018).

[11] Hunter, J. D., "Matplotlib: A 2d graphics environment," *Computing In Science & Engineering* **9**(3), 90–95 (2007).

[12] Soummer, R., Pueyo, L., Sivaramakrishnan, A., and Vanderbei, R. J., "Fast computation of lyot-style coronagraph propagation," *Optics Express* **15**(24), 15935–15951 (2007).

[13] Taylor, G. I., "The spectrum of turbulence," *Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences* , 476–490 (1938).

[14] Lane, R., Glindemann, A., Dainty, J., et al., "Simulation of a kolmogorov phase screen,"

[15] Assémat, F., Wilson, R. W., and Gendron, E., "Method for simulating infinitely long and non stationary phase screens with optimized memory storage," *Optics express* **14**(3), 988–999 (2006).

[16] Ragazzoni, R., "Pupil plane wavefront sensing with an oscillating prism," *Journal of modern optics* **43**(2), 289–293 (1996).

[17] Ragazzoni, R., Diolaiti, E., and Vernet, E., "A pyramid wavefront sensor with no dynamic modulation," *Optics communications* **208**(1-3), 51–60 (2002).

[18] Zernike, F., "Das phasenkontrastverfahren bei der mikroskopischen beobachtung, z," *Tech. Phys* **16**, 454 (1935).

[19] Bloemhof, E. E. and Wallace, J. K., "Phase contrast techniques for wavefront sensing and calibration in adaptive optics," in [*Astronomical Adaptive Optics Systems and Applications*], **5169**, 309–321, International Society for Optics and Photonics (2003).

[20] Sprague, R. A. and Thompson, B. J., "Quantitative visualization of large variation phase objects," *Applied optics* **11**(7), 1469–1479 (1972).

[21] Haffert, S., "Generalised optical differentiation wavefront sensor: a sensitive high dynamic range wavefront sensor," *Optics express* **24**(17), 18986–19007 (2016).

[22] Foo, G., Palacios, D. M., and Swartzlander, G. A., "Optical vortex coronagraph," *Optics letters* **30**(24), 3308–3310 (2005).

[23] Mawet, D., Riaud, P., Absil, O., and Surdej, J., "Annular groove phase mask coronagraph," *The Astrophysical Journal* **633**(2), 1191 (2005).

[24] Mawet, D., Pueyo, L., Carlotti, A., Mennesson, B., Serabyn, E., and Wallace, J. K., "Ring-apodized vortex coronagraphs for obscured telescopes. i. transmissive ring apodizers," *The Astrophysical Journal Supplement Series* **209**(1), 7 (2013).

[25] Codona, J. L. and Angel, R., "Imaging extrasolar planets by stellar halo suppression in separately corrected color bands," *The Astrophysical Journal Letters* **604**(2), L117 (2004).

[26] Kenworthy, M. A., Codona, J. L., Hinz, P. M., Angel, J. R. P., Heinze, A., and Sivanandam, S., "First on-sky high-contrast imaging with an apodizing phase plate," *The Astrophysical Journal* **660**(1), 762 (2007).

[27] Snik, F., Otten, G., Kenworthy, M., Miskiewicz, M., Escuti, M., Packham, C., and Codona, J., "The vector-app: a broadband apodizing phase plate that yields complementary psfs," in [*Modern Technologies in Space-and Ground-based Telescopes and Instrumentation II*], **8450**, 84500M, International Society for Optics and Photonics (2012).

[28] Por, E. H., "Optimal design of apodizing phase plate coronagraphs," in [*Techniques and Instrumentation for Detection of Exoplanets VIII*], **10400**, 104000V, International Society for Optics and Photonics (2017).

[29] Gurobi Optimization, I., "Gurobi optimizer reference manual," (2016).

[30] Carlotti, A., Vanderbei, R., and Kasdin, N., "Optimal pupil apodizations of arbitrary apertures for high-contrast imaging," *Optics Express* **19**(27), 26796–26809 (2011).

[31] Zimmerman, N. T., Riggs, A. E., Kasdin, N. J., Carlotti, A., and Vanderbei, R. J., "Shaped pupil lyot coronagraphs: high-contrast solutions for restricted focal planes," *Journal of Astronomical Telescopes, Instruments, and Systems* **2**(1), 011012 (2016).

[32] Cavarroc, C., Boccaletti, A., Baudoz, P., Fusco, T., and Rouan, D., "Fundamental limitations on earth-like planet detection with extremely large telescopes," *Astronomy & Astrophysics* **447**(1), 397–403 (2006).

[33] Guyon, O., Pluzhnik, E., Kuchner, M., Collins, B., and Ridgway, S., "Theoretical limits on extrasolar terrestrial planet detection with coronagraphs," *The Astrophysical Journal Supplement Series* **167**(1), 81 (2006).

[34] Baudoz, P., Boccaletti, A., Baudrand, J., and Rouan, D., "The self-coherent camera: a new tool for planet detection," *Proceedings of the International Astronomical Union* **1**(C200), 553–558 (2005).

[35] Bos, S. P., Doelman, D. S., De Boer, J., Por, E. H., Norris, B., Escuti, M. J., and Snik, F., "Fully broadband vAPP coronagraphs enabling polarimetric high contrast imaging," in [*Advances in Optical and Mechanical Technologies for Telescopes and Instrumentation III*], *Proc. SPIE* **10706** (2018).

[36] Walt, S. v. d., Colbert, S. C., and Varoquaux, G., "The numpy array: a structure for efficient numerical computation," *Computing in Science & Engineering* **13**(2), 22–30 (2011).

[37] Jones, E., Oliphant, T., and Peterson, P., "{SciPy}: open source scientific tools for {Python}," (2014).

[38] Frigo, M. and Johnson, S. G., "The design and implementation of fftw3," *Proceedings of the IEEE* **93**(2), 216–231 (2005).

[39] Nickolls, J., Buck, I., Garland, M., and Skadron, K., "Scalable parallel programming with cuda," in [*ACM SIGGRAPH 2008 classes*], 16, ACM (2008).

[40] Stone, J. E., Gohara, D., and Shi, G., "Opencl: A parallel programming standard for heterogeneous computing systems," *Computing in science & engineering* **12**(3), 66–73 (2010).

[41] Wang, J. J., Ruffio, J.-B., De Rosa, R. J., Aguilar, J., Wolff, S. G., and Pueyo, L., "pyklip: Psf subtraction for exoplanets and disks," *Astrophysics Source Code Library* (2015).

[42] Gonzalez, C. A. G., Wertz, O., Absil, O., Christiaens, V., Defrère, D., Mawet, D., Milli, J., Absil, P.-A., Van Droogenbroeck, M., Cantalloube, F., et al., "Vip: Vortex image processing package for high-contrast direct imaging," *The Astronomical Journal* **154**(1), 7 (2017).

[43] Konnik, M. and Welsh, J., "High-level numerical simulations of noise in ccd and cmos photosensors: review and tutorial," *arXiv preprint arXiv:1412.4031* (2014).

[44] Hirsch, M., Wareham, R. J., Martin-Fernandez, M. L., Hobson, M. P., and Rolfe, D. J., "A stochastic model for electron multiplication charge-coupled devices–from theory to practice," *PLoS One* **8**(1), e53671 (2013).

[45] Bos, S. P., Haffert, S. Y., and Keller, C. U., "Rigorous vector wave propagation for arbitrary flat media," in [*Polarization Science and Remote Sensing VIII*], **10407**, 1040709, International Society for Optics and Photonics (2017).

[46] Wilby, M. J., Keller, C. U., Snik, F., Korkiakoski, V., and Pietrow, A. G., "The coronagraphic modal wavefront sensor: a hybrid focal-plane sensor for the high-contrast imaging of circumstellar environments," *Astronomy & Astrophysics* **597**, A112 (2017).

[47] Gonsalves, R. A., "Phase retrieval and diversity in adaptive optics," *Optical Engineering* **21**(5), 215829 (1982).